

암호모듈 기본 및 상세 설계서

LEA 블록암호 라이브러리 v1.0

(소프트웨어 암호모듈)

항목	내용
문서 번호	KCMVP-LEA-DS-2025-001
버전 정보	V1.0
작성일	2025년 4월
작성 기관	(주) 한국암호기술
적용 기준	KS X ISO/IEC 19790, 소프트웨어 암호모듈 검증기준

본 문서는 KCMVP 사전 검증 테스트 목적으로 작성된 예시 설계서입니다.

목 차

1장 암호모듈 명세	3
1.1 암호모듈 유형 및 경계	3
1.2 검증대상 암호알고리즘 목록	4
1.3 핵심보안매개변수(CSP) 목록	5
1.4 암호모듈 명칭 및 버전	6
2장 암호모듈 인터페이스	7
2.1 물리적 포트 및 논리적 인터페이스	7
2.2 서비스별 API 명세	8
3장 역할, 서비스 및 인증	10
3.1 운영자 역할 정의	10
3.2 서비스 목록 및 접근 권한	11
4장 운영환경	12
5장 암호 키 관리	13
5.1 키 생성	13
5.2 키 저장 및 배포	14
5.3 키 제로화	15
6장 자가시험	16
7장 설계보증 및 형상관리	18

1장. 암호모듈 명세

1.1 암호모듈 유형 및 경계

본 암호모듈은 소프트웨어 암호모듈로 분류된다. 소프트웨어 암호모듈이란 암호경계 내에 하나 또는 그 이상의 소프트웨어 컴포넌트만으로 구성된 모듈을 의미하며, 변경 가능한 운영환경에서 동작한다.

암호모듈 유형 선택 근거:

- 물리적 하드웨어 컴포넌트 없이 순수 C 라이브러리(.so, .dll) 형태로 배포
- 범용 운영체제(Linux, Windows) 위에서 동작하는 변경 가능한 운영환경
- 암호경계는 lea_lib.so 파일 단일 공유 라이브러리로 정의

암호경계 구성요소:

구성요소	파일명	유형	설명
암호 코어	lea_core.c	소스코드	LEA 블록암호 라운드 함수
키 스케줄	lea_key.c	소스코드	128/192/256비트 키 확장
모드 드라이버	lea_mode.c	소스코드	CBC/CTR/GCM/CCM 모드
공개 API	lea.h	헤더	외부 노출 인터페이스
타입 정의	lea_types.h	헤더	uint32_t 등 타입 정의

1.2 검증대상 암호알고리즘 목록

본 암호모듈이 구현하는 검증대상 암호알고리즘 목록은 다음과 같다.

알고리즘	동작모드	키 길이(비트)	표준 참조	검증대상 여부
LEA	CBC	128/192/256	KS X 3246	검증대상
LEA	CTR	128/192/256	KS X 3246	검증대상
LEA	GCM	128/192/256	KS X 3246	검증대상
LEA	CCM	128/192/256	KS X 3246	검증대상
LEA	ECB	128/192/256	KS X 3246	검증대상 (테스트 전용)
DRBG	CTR	256	KS X 3186	검증대상

비검증대상 암호알고리즘: 본 암호모듈은 비검증대상 암호알고리즘을 포함하지 않는다. 모든 암호 연산은 검증대상 동작모드로만 수행된다.

1.3 핵심보안매개변수(CSP) 목록

핵심보안매개변수(CSP: Critical Security Parameter)는 노출 또는 수정 시 암호모듈의 보안을 위협할 수 있는 보안 관련 정보이다.

CSP 명칭	유형	크기	생성 방법	소멸 방법
비밀키(MK)	비밀키	128/192/256비트	DRBG 생성	제로화
라운드키(RK)	파생키	32×6×32비트	키 스케줄	제로화
CBC IV	초기벡터	128비트	DRBG 생성	사용 후 제로화
CTR 카운터	상태값	128비트	초기화 시 설정	제로화
GCM 인증태그	인증값	96~128비트	GHASH 연산	출력 후 제로화
DRBG 내부상태	CSP	384비트	엔트로피 시드	제로화

CSP 생성·소멸 정책: 모든 CSP는 사용 완료 후 메모리에서 제거하여 이후 접근이 불가하도록 관리한다. 구체적인 제로화 방법은 5장에서 기술한다.

1.4 암호모듈 명칭 및 버전

항목	내용
암호모듈 명칭	LEA 블록암호 라이브러리
버전 정보	V1.0 (2025.04 기준)
라이브러리 파일	lea_lib.so (Linux), lea_lib.dll (Windows)
버전 부여 방식	Major.Minor (예: V1.0, V1.1, V2.0)
식별자	KCMVP-LEA-2025-001

2장. 암호모듈 인터페이스

2.1 물리적 포트 및 논리적 인터페이스

본 소프트웨어 암호모듈은 물리적 포트를 갖지 않으며, 운영체제의 프로세스 API 경계를 논리적 인터페이스로 사용한다.

논리적 인터페이스	방향	설명
데이터 입력(DI)	→ 모듈	평문, 키 소재, IV, AAD 등
데이터 출력(DO)	← 모듈	암호문, 복호화 평문, 인증태그
제어 입력(CI)	→ 모듈	알고리즘 선택, 키 길이, 모드 설정
상태 출력(SO)	← 모듈	반환코드(0=성공, 음수=오류), 오류 메시지

2.2 서비스별 API 명세

암호모듈이 제공하는 공개 API 목록과 각 API의 데이터 입출력 및 제어 입력을 명세한다.

▶ `lea_set_key()`

항목	내용
파라미터	<code>uint32_t RK[32][6]</code> (출력), <code>const uint8_t *mk</code> (입력), <code>int mk_len</code> (제어)
반환값	<code>void</code> (성공) / 없음
설명	키 스케줄 수행 — 비밀키 <code>mk</code> 로부터 라운드키 <code>RK</code> 생성

▶ `lea_cbc_enc()`

항목	내용
파라미터	<code>uint8_t *ct</code> (출력), <code>const uint8_t *pt</code> (입력), <code>int len</code> , <code>const uint8_t *iv</code> (입력), <code>const uint32_t RK[32][6]</code> (입력)
반환값	<code>int</code> (0=성공, -1=오류)
설명	CBC 모드 암호화

▶ `lea_cbc_dec()`

항목	내용
파라미터	<code>uint8_t *pt</code> (출력), <code>const uint8_t *ct</code> (입력), <code>int len</code> , <code>const uint8_t *iv</code> , <code>const uint32_t RK[32][6]</code>
반환값	<code>int</code> (0=성공, -1=오류)
설명	CBC 모드 복호화

▶ `lea_ctr_enc()`

항목	내용
파라미터	<code>uint8_t *out</code> (출력), <code>const uint8_t *in</code> (입력), <code>int len</code> , <code>uint8_t *ctr</code> (입출력), <code>const uint32_t RK[32][6]</code>
반환값	<code>int</code> (0=성공, -1=오류)

항목	내용
설명	CTR 모드 암호/복호화

▶ `lea_gcm_enc()`

항목	내용
파라미터	<code>uint8_t *ct</code> , <code>uint8_t *tag</code> (출력), <code>const uint8_t *pt</code> (입력), <code>int pt_len</code> , <code>const uint8_t *iv</code> , <code>const uint8_t *aad</code>
반환값	<code>int</code> (0=성공, -1=오류)
설명	GCM 모드 인증 암호화

3장. 역할, 서비스 및 인증

3.1 운영자 역할 정의

본 암호모듈은 라이브러리 형태로 제공되므로 KS X ISO/IEC 19790에 따라 사용자 역할이 관리자 역할을 겸한다.

역할	설명	인증 방법
사용자	암호 서비스(암/복호화, 키 생성) 수행	운영체제 프로세스 권한
관리자	초기화, 자가시험, 제로화 수행	운영체제 프로세스 권한

3.2 서비스 목록 및 접근 권한

서비스명	역할	CSP 접근	설명
초기화	관리자	RW	자가시험 수행 후 동작 모드 진입
키 생성	사용자	W	lea_set_key() 호출
암호화	사용자	R	CBC/CTR/GCM 암호화
복호화	사용자	R	CBC/CTR/GCM 복호화
제로화	사용자/관리자	W	CSP 메모리 소거
자가시험	관리자	R	KAT 수행 및 무결성 검사
상태 조회	사용자	-	현재 동작 상태 반환

4장. 운영환경

4.1 지원 운영환경

본 암호모듈은 변경 가능한 운영환경에서 동작하는 소프트웨어 암호모듈이다. 지원 운영환경은 다음과 같다.

운영체제	버전	비트	컴파일러
Linux (Ubuntu)	20.04 LTS 이상	64비트	GCC 9.x 이상
Linux (CentOS)	7.x 이상	64비트	GCC 4.8 이상
Windows	10/11	64비트	MSVC 2019 이상
macOS	12.x 이상	64비트	Apple Clang 13 이상

단일 운영자 동작모드: 암호모듈이 라이브러리 형태로 응용프로그램에 탑재되어 해당 응용프로그램이 단일 운영자 역할을 수행한다.

메모리 보호: 운영체제의 가상 메모리 분리, ASLR(Address Space Layout Randomization), DEP(Data Execution Prevention) 등의 메모리 보호 메커니즘을 활용하여 외부 프로세스로부터의 CSP 접근을 방지한다.

5장. 암호 키 관리

5.1 키 생성

비밀키는 반드시 검증대상 DRBG(CTR-DRBG, KS X 3186 준거)를 통해 생성한다.

키 종류	생성 방법	엔트로피 소스
비밀키(MK)	CTR-DRBG 출력	/dev/urandom, CryptGenRandom
CBC IV	CTR-DRBG 출력	/dev/urandom, CryptGenRandom
CTR 초기값	CTR-DRBG 출력 또는 설정	응용프로그램 제공 허용

5.2 키 저장 및 배포

비밀키는 암호모듈 내부 메모리에만 존재하며, 외부 저장매체에 평문으로 기록되지 않는다.

- 키 저장: 운영 중 라운드키(RK)는 프로세스 가상 메모리 내 동적 할당 영역에 존재
- 키 배포: 검증대상 키 설정 메커니즘(직접 전달) 사용, 네트워크 전송 시 암호화 필수
- 키 수명: 비밀키는 응용프로그램의 세션 단위로 유효, 세션 종료 시 제거

5.3 키 제로화

사용이 완료된 CSP는 재사용이 불가능하도록 즉시 제거해야 한다.

제로화 대상:

- 비밀키(MK): 라운드키 생성 완료 후 즉시 제거
- 라운드키(RK): 암호/복호화 완료 후 또는 세션 종료 시 제거
- IV/카운터: 사용 완료 후 제거

제로화 구현:

메모리 초기화는 표준 메모리 초기화 함수를 사용하여 해당 영역을 0으로 덮어쓴다. 구현 시 컴파일러 최적화에 의해 제로화 코드가 제거되지 않도록 주의하여 구현한다.

제로화 확인: 제로화 수행 후 해당 메모리 영역이 0으로 초기화되었는지 자가시험을 통해 확인한다.

6장. 자가시험

6.1 전원인가 자가시험 (Power-Up Self-Test)

암호모듈은 초기화 시 다음 전원인가 자가시험을 수행한다. 자가시험 중에는 암호 서비스가 제공되지 않으며 데이터 출력이 금지된다.

시험 항목	시험 방법	대상 알고리즘
LEA-CBC KAT	알려진 평문/암호문 비교	LEA-CBC
LEA-CTR KAT	알려진 평문/암호문 비교	LEA-CTR
LEA-GCM KAT	알려진 평문/인증태그 비교	LEA-GCM
LEA-CCM KAT	알려진 평문/인증태그 비교	LEA-CCM
CTR-DRBG KAT	알려진 출력값 비교	CTR-DRBG
소프트웨어 무결성	HMAC-SHA-256 해시 비교	전체 모듈

6.2 조건부 자가시험 (Conditional Self-Test)

- 연속 난수 생성 시험: DRBG에서 연속으로 동일한 값이 출력될 경우 오류 처리
- 소프트웨어 로드 시험: 외부 컴포넌트 로드 시 무결성 검증

6.3 자가시험 실패 처리

자가시험 실패 시 암호모듈은 심각한 오류 상태(Error State)로 진입하며, 이후 모든 암호 서비스 요청에 대해 오류 코드를 반환하고 암호 연산을 수행하지 않는다.

오류 코드	의미	해제 방법
LEA_ERR_KAT	KAT 실패 — 알고리즘 오류	암호모듈 재설치
LEA_ERR_INT	무결성 검증 실패	암호모듈 재설치
LEA_ERR_RNG	DRBG 연속 중복 출력	암호모듈 재시작

※ 암호모듈의 동작 상태는 초기화 상태, 동작 상태, 오류 상태로 구분되며 각 상태 간 전이는 API 호출 및 자가시험 결과에 의해 결정된다.

7장. 설계보증 및 형상관리

7.1 형상관리

본 암호모듈의 소스코드, 설계 문서, 시험서 등 모든 형상항목은 Git 형상관리 시스템으로 관리된다.

형상항목	식별 방법	관리 도구
소스코드	Git 커밋 해시	Git
설계서	문서 버전 V1.0, V1.1 ...	Git + 문서 버전
시험서	시험 버전 T1.0, T1.1 ...	Git + 문서 버전
실행 라이브러리	SHA-256 해시값	빌드 시스템

7.2 버전 부여 방법

버전은 Major.Minor 형식으로 부여한다.

- Major 버전: 암호알고리즘 변경, 보안 요구사항 변경 등 중요 변경 시 증가
- Minor 버전: 버그 수정, 운영환경 추가, 문서 보완 등 경미한 변경 시 증가
- 각 형상항목의 변경 이력은 Git 로그로 추적하며 변경 시마다 커밋 메시지에 변경 내역을 기록한다.

7.3 개발 보증

본 암호모듈은 다음 설계 원칙을 준수하여 개발되었다.

- 최소 권한 원칙: 각 함수는 필요한 최소한의 CSP에만 접근
- 방어적 프로그래밍: 입력 유효성 검증 후 연산 수행
- 버퍼오버플로우 대응: 경계 검사 강화, 안전한 메모리 함수 사용

본 설계서는 KS X ISO/IEC 19790 및 소프트웨어 암호모듈 검증기준에 따라 작성되었습니다. 검증 신청 전 시험기관의 사전 검토를 받으시기 바랍니다.